

# Omni-Compass: mechanism of action, from the equations to the muscles

Every statement here is tied to code in this repository, and the measured numbers are reproduced by `python tools/mechanism.py`, which writes `results/MECHANISM_OF_ACTION.json`. The engine files are frozen. Their SHA-256 hashes are checked by `verify.py`.

## 1. The state: six numbers that describe the whole system

| Symbol   | Name                | What it stands for in a data centre                                             |
|----------|---------------------|---------------------------------------------------------------------------------|
| E        | energy / excitation | how hard the system is being driven: queues, overload, power, heat              |
| U        | order parameter     | how healthy and settled the system is: +1 healthy basin, -1 failed basin        |
| I_U      | integrated need     | unmet need accumulated over time: memory of pressure that has not been relieved |
| S        | stress              | slow structural stress from heat, power, network, security, staleness           |
| B, B_dot | bath and its rate   | a damped oscillator driven by stress: the slow "tide" of physical load          |

## 2. The equations (omnicompass/core.py, lines 5-12)

```
(1) dE/dt = -alpha_E E + beta_int + beta_ext + v_eff
(2) dU/dt = mu U (1 - U^2) - (dE/dt)/E_max - lambda_U U + u,          |u| <= 25
(3) dI_U/dt = (1 - U) - sigma_1 E - delta S - lambda_I I_U
(4) v_eff = cos(omega_B t / 2) * c * tanh(lambda_0 + lambda_1 (U - 0.5) + lambda_2 S)
(5) Phi(S) = alpha_s S^2 / 2 + beta_s S^3 / 4 - delta S
(6) dS/dt = -dPhi/dS = delta - alpha_s S - (3/4) beta_s S^2
(7) dB/dt = B_dot ; dB_dot/dt = gamma_c delta S - (omega_B / Q_B) B_dot - omega_B^2 B
(8) R_B[n] = finite-difference audit of (7), never fed back
```

Each term does one job:

- **(1) Energy.** Energy decays at rate  $\alpha_E = 3.487$ . It is driven by two forcings: internal pressure  $\beta_{int}$  (queue and overload) and external pressure  $\beta_{ext}$  (power, heat, network, staleness, security). The drive term  $v_{eff}$  (4) is bounded by  $c$  through the hyperbolic tangent, so no input can drive  $E$  without limit. This is the saturation (the  $\tanh$  "speed limit"): however extreme the observation, the engine's response rate is capped at  $c = 1$ .
- **(2) Order.**  $\mu U (1 - U^2)$  is a double-well (bistable) drift with two stable basins,  $U = +1$  (healthy) and  $U = -1$  (failed), and an unstable ridge at  $U = 0$ . A rise in energy ( $dE/dt$ ) pushes  $U$  out of the healthy basin.  $u$  is the bounded control input.
- **(3) Integrated need.**  $I_U$  grows while  $U$  is below 1 (the system is not settled). It is reduced by energy and stress already being dealt with, and it leaks at  $\lambda_I = 0.719$ . It is the engine's memory of unrelieved pressure.
- **(5)-(6) Stress.** A cubic potential (the "well in the basin").  $S$  rolls downhill to its stable root. This is the slowest restoring force.

- **(7) Bath.** A damped harmonic oscillator (natural frequency  $\omega_B = 1$ , quality  $Q_B = 2.5$ ) driven by stress. It turns stress into a smooth, lagged "tide". It is the only second-order element.
- **(8) Audit.** The bath equation is re-checked numerically every step. The check is never fed back.

Integration: classical 4th-order Runge-Kutta, 10 micro-steps of 0.01 per macro step (rk4\_step, macro\_step). The C++ twin matches the Python to 3.6e-15 on 500 reference trajectories.

### 3. From telemetry to state (omnicompass/adapter.py, observe\_vector, assimilate)

Each decision the telemetry is normalised to nine observations: q (queue), load, power, thermal, network, drift, stale, security, conflict. An observed state is computed from them:

```
E_obs = 0.25 q + 0.18 max(0, load - 0.85) + 0.16 power + 0.13 thermal + 0.10 network + 0.08 drift + 0.06 conflict + 0.04 stale
U_obs = 1 - (0.27 q + 0.18 power + 0.16 thermal + 0.12 network + 0.12 drift + 0.10 conflict + 0.05 stale)
S_obs = 0.36 thermal + 0.28 power + 0.18 network + 0.10 security + 0.08 stale - 0.20 q
B_obs = 0.52 power + 0.26 thermal + 0.22 drift
I_obs = q + drift + conflict
beta_int = 0.58 q + 0.42 max(0, load - 0.75)      beta_ext = 0.33 power + 0.27 thermal + 0.20 network + 0.12 stale + 0.08 security
```

The state is blended toward the observed state with weight  $a = 0.339$ :  $x \leftarrow (1 - a) x + a x_{\text{obs}}$ . It is then evolved by one macro step (0.1 engine time units) of equations (1)-(7).

### 4. From state to action (the allocation law, Governor . step)

```
push      = u(x) / 25,  u(x) = clip(-f_U(x) + 12 (1 - U), -25, 25)      equation (2)'s controller, evaluated, not applied
rho*      = clamp(rho0 - kI I_U - kE E, rho_min, rho0)                target utilisation (the HPA target)
n_req     = ceil(n load / rho*) + ceil(kq q n)                        machines wanted
release   only while push <= push_release (the engine reports convergence), after band and dwell
cap       = clamp(cap_now load (1 + margin), cap_min, 1) when calm and U >= U_gate; 1 under backlog;
           always <= 1 - cap_gain max(0, B - B_cap)                  the bath limits the power cap
change    permitted only while U >= U_gate and no security block
rollback  authorised if S >= S_rollback, U is falling, or a security block is seen
```

### 5. Which equation drives which muscle

| Muscle                                               | Engine quantity used                                                                     | Where                                                       |
|------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| HPA target (pods)                                    | $\rho^*$ from $I_U$ and $E$ , equations (1) and (3)                                      | adapter Governor . step; live omni_controller/controller.py |
| Node pool (machines)                                 | $n_{\text{req}}$ from load, queue and $\rho^*$ ; release gated by push from equation (2) | same                                                        |
| Power cap (pod CPU limits, CPU frequency, GPU limit) | cap from U gate and bath B, equation (7)                                                 | same; omni_controller/muscles.py                            |
| Rollout guard                                        | change_permitted (U), rollback (S, dU)                                                   | muscles.py                                                  |
| Security hold                                        | security observation raises S and blocks expansion                                       | adapter, shield                                             |
| Speed-first law                                      | $\rho^*$ for pods; push gates machine release                                            | omnicompass/speed.py                                        |

| Muscle            | Engine quantity used                                            | Where                  |
|-------------------|-----------------------------------------------------------------|------------------------|
| Right-sizing      | rho* sizes CPU; E widens the memory margin; push gates scale-in | omnilab/rightsize.py   |
| Cold start        | rho* sizes instances; I_U sets how long to keep warm            | omnilab/coldstart.py   |
| GPU packing       | I_U pre-warms nodes; push gates power-off                       | omnilab/gpupack.py     |
| Training power    | E tightens the applied ramp limit under sustained grid stress   | omnilab/powersmooth.py |
| GPU health        | S and E shorten the checkpoint interval                         | omnilab/health.py      |
| Cooling           | E widens the inlet-temperature margin                           | omnilab/cooling.py     |
| Inference         | the full allocation law (node_delta) sizes replicas             | omnilab/inference.py   |
| Agent containment | I_U throttles; the security observation contains                | omnilab/containment.py |
| VM energy         | I_U and E raise the learning rate when attribution goes stale   | omnilab/vmenergy.py    |

## 6. Measured: what the equations actually do inside the governor

Measured on a real observation stream (fleet plant, web services, fleet mode, 360 decisions):

| Mode (eigenvalue of the Jacobian at the operating point) | Time constant   | In decisions                 |
|----------------------------------------------------------|-----------------|------------------------------|
| -7.90                                                    | 0.13            | 1.3                          |
| -3.10                                                    | 0.32            | 3.2                          |
| -0.72                                                    | 1.39            | 13.9                         |
| -0.20 +/- 0.98i (the bath)                               | 5.0, period 6.4 | 50, rings every 64 decisions |
| -0.17                                                    | 6.0             | 60                           |

- **Every mode is stable** (all real parts negative). The operating point is a stable equilibrium inside the healthy basin:

E = 0.095, U = 0.976, S = 0.315, B = 0.10. This is the basin keeping everything in, stated as mathematics.

- **Per decision, the equations and the observation move the state about equally.** Across all six variables the

equations contribute 45-50% of each step's movement and the observation blend the rest.

- **Equation (2)'s control input u is not applied during governance.** The engine evolves open-loop (u = 0). The control law is evaluated only to produce push, which decides when capacity may be released.

- **Compute:** about 95 microseconds per decision in Python, and about 2.9 microseconds in the C++ twin.

## 7. What this explains, and what it points to

- **Why "engine off" scores close to "engine on" in the muscle benchmarks.** With the equations frozen, the observation

blend still moves the state. Both versions are then low-pass filters of the same telemetry. The engine's fast modes (1-3 decisions) behave almost like the blend itself. Its distinct contribution is its slow memory: I\_U (14 decisions), S (60) and the bath (period 64 decisions). The muscle mappings mostly read E and  $\rho^*$ , which live on the fast modes.

- **The engine clock is not matched to the workload's rhythm.** One decision advances engine time by 0.1. At 60-second

decisions the bath rings every 64 minutes, but web demand has a 24-hour rhythm and training power a 2-second one. This is a wiring choice in the adapter (engine time per decision), not a property of the equations.

- **The controller in equation (2) is unused as an actuator.** Wiring its output  $u$  (or  $U$ 's trajectory under  $u$ ) directly

to the actuators is the unexplored way to "let the math do the work".

Three wiring experiments follow directly. None changes a single equation:

1. **Clock matching.** Set engine time per decision so that the bath's natural period equals the workload's rhythm. The oscillator in (7) then carries the cycle and can anticipate it.
2. **Slow-mode actuation.** Drive machines from I\_U, S and B (the memory modes) rather than E (a fast mode).
3. **Closed-loop actuation.** Apply equation (2)'s  $u$  and use its magnitude, not only its sign, as the actuator command.

Each is testable in the existing harness against every platform in the league (tuning/league.py), and will be tested there.